DATABYT · HANDS-ON BUILD MANUAL

Support Agent — The Build Manual

A complete, copy-paste, do-it-without-help guide. Every command, every file in full, every screen you'll see, and every error with its fix.

This manual assumes you are alone. No Claude Code, no searching needed. Follow it top to bottom, pass every ✅ checkpoint, and you'll have a live AI support agent answering customers inside DataByt.

Windows / PowerShell · Python ADK · Gemini Flash · Supabase · Cloud Run · Next.js | ~1 focused day

How this manual works

You build in **7 parts**. Each ends with a ✅ **CHECKPOINT** — something you can see that proves it worked. **Don't move on until the checkpoint passes.** If it fails, the 🔧 **IF IT BROKE** box tells you exactly what to do.

Part	What you build	Time	You'll see
0	Setup	15 min	tools installed
1	Hello Agent	20 min	an agent talks to you in a browser
2	Agent + Tool	30 min	it calls a function, uses the result
3	Knowledge Agent	30 min	it answers "how do I..." questions
4	The Manager (multi-agent)	45 min	it routes tickets to 3 specialists
5	Real DataByt Data	45 min	it reads your actual Supabase invoices
6	Deploy to the Internet	45 min	it's live on a public URL
7	Wire into DataByt	30 min	dashboard → agent → reply

PART 0

Setup (do this once)

15 minutes

0.1 — Check Python

```
python --version          # expect: Python 3.11.x  (any 3.10+ ok)
```

🔧 **IF IT BROKE** "not recognized" → install from python.org, **tick "Add Python to PATH"**, reopen PowerShell.

0.2–0.4 — Folder, venv, install

```
cd d:\AI\DP\databyt-saas
mkdir agents
cd agents
python -m venv .venv
.venv\Scripts\Activate.ps1     # prompt should now show (.venv)
pip install google-adk         # ends: Successfully installed google-adk-...
```

> 🔧 **IF IT BROKE** "running scripts is disabled": run once → `Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned`, then activate again.

0.5 — Get your free Gemini key

Open **aistudio.google.com/apikey** → sign in → **Create API key** → copy it (looks like `AIzaSy...`).

> ✅ **CHECKPOINT 0**
> `adk --version` → shows a version like `1.x.x`. If yes, Part 0 done.
> 🔧 **If "adk not recognized":** venv isn't active — re-run `.venv\Scripts\Activate.ps1` (prompt must show `(.venv)`).

PART 1

Hello Agent

20 minutes · the simplest agent, talking in a browser

```
mkdir support_agent
cd support_agent
```

Create **three files** inside `support_agent/`:

__init__.py

```
from . import agent
```

.env (paste your real key, no quotes, no spaces)

```
GOOGLE_GENAI_USE_VERTEXAI=FALSE
GOOGLE_API_KEY=AIzaSy_paste_your_real_key_here
```

agent.py

```
from google.adk.agents import Agent

root_agent = Agent(
    name="support_agent",
    model="gemini-2.0-flash",
    description="DataByt customer support assistant.",
    instruction="""
    You are DataByt's friendly support assistant. DataByt automates
    accounts-receivable collections for finance teams. Greet warmly,
    answer clearly and concisely. If unsure, say so — never make up facts.
    """,
)
```

Go UP to the `agents` folder and launch the UI:

```
cd ..
adk web                 # expect: Uvicorn running on http://127.0.0.1:8000
```

✅ CHECKPOINT 1

Open **localhost:8000** → pick `support_agent` in the dropdown → type `Hi, what is DataByt?` → it replies with a friendly explanation. **You just built and ran your first AI agent.** 🎉

🔧 IF IT BROKE

- **Dropdown empty** → you ran `adk web` from the wrong folder. Be in `agents` (which *contains* support_agent). Ctrl-C, re-run.
- **API key / 403 error** → `.env` key wrong. Re-copy. Line must be exactly `GOOGLE_API_KEY=AIza...`
- **ModuleNotFoundError** → venv not active. Ctrl-C → activate → re-run.

Stop the server any time with **Ctrl-C**.

PART 2

Agent + Tool (giving it hands)

30 minutes · fake data, works instantly

Stop the server. Replace `support_agent/agent.py`:

```
from google.adk.agents import Agent

def get_invoice_count(status: str) -> dict:
    """Returns how many invoices have a given status.
    Args:
        status: One of 'open', 'overdue', or 'paid'.
    """
    fake = {"open": 12, "overdue": 5, "paid": 240}
    if status not in fake:
        return {"status": "error", "message": f"Unknown status '{status}'."}
    return {"status": "success", "count": fake[status]}

root_agent = Agent(
    name="support_agent", model="gemini-2.0-flash",
    description="Support assistant that can check invoice counts.",
    instruction="""When asked how many invoices are open/overdue/paid,
    ALWAYS use get_invoice_count. Never guess. Report it in a friendly sentence.""",
    tools=[get_invoice_count],
)
```

✅ CHECKPOINT 2

`adk web` → ask `How many invoices are overdue?` → it replies **5**. Click the **Events** panel → you see it called `get_invoice_count(status="overdue")` and got `count: 5` . **That panel is how you debug everything from now on.**

🔧 IF IT BROKE Made up a number without the tool → docstring unclear or instruction too soft. Keep the exact docstring; the agent reads it to know what the tool does.

PART 3

Knowledge Agent (answers "how do I...")

30 minutes · knowledge lives in the instruction (simple, works now)

```
mkdir knowledge_agent
cd knowledge_agent
copy ..\support_agent\.env .env       # reuse the same key
```

`__init__.py` → `from . import agent`. Then `agent.py`:

```
root_agent = Agent(
  name="knowledge_agent", model="gemini-2.0-flash",
  description="Answers how-to and product questions about DataByt.",
  instruction="""Answer using ONLY this knowledge. If not here, say you're
  not sure and offer a human. Never invent steps.
  - CONNECT QB/XERO: Dashboard > Integrations > Connect, log in, Allow. ~5 min.
  - DUNNING: one email per customer, tone L1/L2/L3 by days late.
    Change thresholds in Settings > Collection Rules.
  - PAYMENT LINKS: set your own in Settings > Payment Collection.
  - DISPUTES: file on AR Aging; collections pause; resolve to resume.
  - REPORTS: Dashboard > Reports > Generate PDF Report.
  - PRICING: one plan, founding rate locked for first 20, 30-day free trial.""",
)
```

CHECKPOINT 3

Pick `knowledge_agent` → ask `How do I connect QuickBooks?` → gives the steps. Ask `What is the capital of France?` → it says it's not sure / offers a human. **That "I don't know" is correct** — it won't lie to customers.

PART 4

The Manager (multi-agent routing)

45 minutes · the heart of the system

Replace `support_agent/agent.py` with the full manager + 3 specialists (the manager routes; specialists do the work):

```
# --- tool for the account specialist (fake for now) ---
def get_invoice_count(status: str) -> dict:
    """Returns count of invoices with a status. Args: status: open/overdue/paid."""
    fake = {"open": 12, "overdue": 5, "paid": 240}
    return {"status":"success", "count": fake.get(status, 0)}

knowledge_agent  = Agent(name="knowledge_agent", model="gemini-2.0-flash",
  description="Answers how-to/product questions.", instruction="...(knowledge)...")

account_agent    = Agent(name="account_agent", model="gemini-2.0-flash",
  description="Checks invoice/account data.",
  instruction="Call get_invoice_count for counts. Never guess.",
  tools=[get_invoice_count])

escalation_agent = Agent(name="escalation_agent", model="gemini-2.0-flash",
  description="Refunds, billing disputes, bugs, upset customers.",
  instruction="Apologise briefly, say a human will follow up shortly.")

root_agent = Agent(name="support_manager", model="gemini-2.0-flash",
  instruction="""Route each message:
    'How do I...', product/pricing  -> knowledge_agent
    'How many invoices...', status   -> account_agent
    refunds/disputes/bugs/anger/unclear -> escalation_agent
    Always route. Don't answer specialist questions yourself.""",
  sub_agents=[knowledge_agent, account_agent, escalation_agent])
```

Full code is in `build-manual.md` (the specialist instructions are abbreviated above to fit the page — copy them in full from the markdown file).

✅ CHECKPOINT 4

Pick `support_agent` (now the manager). Send 3 messages, watch **Events** each time:

- `How do I connect QuickBooks?` → transfers to **knowledge_agent**
- `How many invoices are overdue?` → **account_agent** → tool → **5**
- `I want a refund` → **escalation_agent** → human follow-up

All three route right → **you built a real multi-agent system.**

🔧 **IF IT BROKE** Everything goes to one specialist → make each specialist's `description` clearly state what it handles (the manager uses descriptions to decide). Answers without transferring → strengthen "Always route."

PART 5

Real DataByt Data (read live invoices)

45 minutes · replace fake data with your Supabase

```
pip install supabase
```

Add to `support_agent/.env` (get values from Vercel env vars or Supabase → Settings → API → **service_role** key):

```
SUPABASE_URL=https://yourproject.supabase.co
SUPABASE_SERVICE_KEY=your_service_role_key
```

Replace ONLY the `get_invoice_count` function:

```
import os
from supabase import create_client
_sb = create_client(os.environ["SUPABASE_URL"], os.environ["SUPABASE_SERVICE_KEY"])

def get_invoice_count(status: str, org_id: str) -> dict:
    """Count invoices for an org by status. Args: status, org_id."""
    try:
        res = (_sb.table("invoices").select("id", count="exact")
               .eq("org_id", org_id).eq("status", status).execute())
        return {"status":"success", "count": res.count or 0}
    except Exception as e:
        return {"status":"error", "message": str(e)}
```

✅ CHECKPOINT 5

`adk web` → ask `How many overdue invoices does org abc-123 have?` (use a real org_id from your `organizations` table) → returns the **real** count from your database.

🔧 **IF IT BROKE** `KeyError: SUPABASE_URL` → env line missing in `support_agent/.env`. Auth error → you used the anon key; use the **service_role** key.

PART 6

Deploy to the Internet (Cloud Run)

45 minutes · scales to zero = near-free

Install **Google Cloud CLI** from cloud.google.com/sdk/docs/install (Windows installer, defaults). Reopen PowerShell, reactivate venv. Then:

```
gcloud auth login                          # browser opens, sign in
gcloud config set project YOUR_PROJECT_ID
gcloud services enable run.googleapis.com aiplatform.googleapis.com
```

Create `support_agent/requirements.txt`:

```
google-adk
supabase
```

Deploy (from the `agents` folder):

```
adk deploy cloud_run --project YOUR_PROJECT_ID --region us-central1 .\support_agent
# when asked "Allow unauthenticated invocations?" type:  y
```

Set the production env vars (Cloud Run can't read your local `.env`):

```
gcloud run services update support-agent --region us-central1 `
  --set-env-vars GOOGLE_API_KEY=AIza...,SUPABASE_URL=https://...,SUPABASE_SERVICE_KEY=...
```

✅ **CHECKPOINT 6**

Deploy prints `Service URL: https://support-agent-xxxx-uc.a.run.app` — copy it. In a new PowerShell: `curl https://...run.app/list-apps` → JSON containing `support_agent`. **Your agent is live on the internet.** 🎉

🦴 **IF IT BROKE** Billing error → console.cloud.google.com → Billing → link your credits to the project. Build failed → ensure `requirements.txt` exists with `google-adk` and `supabase`, redeploy.

PART 7

Wire It Into DataByt

30 minutes · TypeScript, in your existing app

In Vercel → Settings → Environment Variables, add `SUPPORT_AGENT_URL` = your Cloud Run URL (no trailing slash). Then create `src/app/api/support/route.ts`:

```
export async function POST(req: NextRequest) {
  let body; try { body = await req.json(); }
  catch { return NextResponse.json({ error:"Invalid body" }, { status:400 }); }
  const { message, orgId, ticketId } = body;
  if (!message) return NextResponse.json({ error:"message required" }, { status:400 });

  const sid = ticketId ?? crypto.randomUUID();
  const uid = orgId ?? "anonymous";
  // 1) create a session
  await fetch(`${AGENT_URL}/apps/support_agent/users/${uid}/sessions/${sid}`,
    { method:"POST", headers:{"Content-Type":"application/json"}, body:"{}" }).catch(()=>{});
  // 2) run the agent
  const r = await fetch(`${AGENT_URL}/run`, { method:"POST",
    headers:{"Content-Type":"application/json"},
    body: JSON.stringify({ app_name:"support_agent", user_id:uid,
      session_id:sid, new_message:{ role:"user", parts:[{ text:message }] } }) });
  const events = await r.json();
  const last = Array.isArray(events) ? events[events.length-1] : events;
  const reply = last?.content?.parts?.map((p)=>p.text).join("") ?? "Sorry…";
  return NextResponse.json({ reply });
}
```

Full route (with imports + the `AGENT_URL` const) is in `build-manual.md` .

✅ CHECKPOINT 7

Commit + push (Vercel deploys). Then test live:

```
curl -Method POST https://www.databyt.in/api/support `
  -Headers @{ "Content-Type"="application/json" } `
  -Body '{"message":"I want a refund","orgId":"test","ticketId":"t2"}'
```

Sensible reply (escalation) → **your agent is in production, end to end: DataByt → Cloud Run → Gemini → reply.** 🎉 🎉

🦴 **IF IT BROKE** 502 → `SUPPORT_AGENT_URL` wrong/missing in Vercel (no trailing slash). **Empty reply** → `console.log(JSON.stringify(events))` , look at the shape, adjust the last-event extraction. The text is always in some event's `content.parts[].text` .

PART 8 & 9

Troubleshooting Bible + Going Solo

The universal 4-step debug

1. **Read the actual error's LAST line.** The fix is usually named there.
2. **Is the venv active?** Prompt must show `(.venv)` . 80% of "it suddenly broke" is this.
3. **Right folder?** `adk web` runs from the folder that *contains* your agent folders.
4. **Did you save the file?** Edits don't count until saved; ADK auto-reloads on save.

Error contains	Fix
`adk is not recognized`	activate venv: `.venv\Scripts\Activate.ps1`
`ModuleNotFoundError: google`	`pip install google-adk`
`API key not valid` / 403	re-copy key into `.env`
dropdown empty	`cd` to the parent of your agent folder
`KeyError: SUPABASE_URL`	add it to that agent's `.env`
agent invents data	add "always use the tool / never guess"
Cloud Run billing error	console → Billing → link project
`502` from /api/support	fix `SUPPORT_AGENT_URL` in Vercel

When you're stuck without help

- **Copy the exact error line into Google.** First result usually has it.
- **Official docs:** `google.github.io/adk-docs` — Get Started + Tutorials mirror this manual.
- **Change one thing at a time.** Never change three then run.
- **The Events panel is truth.** It shows exactly which tool the agent called with what args. Read it before guessing.

🏁 WHAT YOU'LL HAVE BUILT

An AI support agent that: runs in a visual UI locally · routes tickets to 3 specialists · reads your REAL Supabase invoices · is live on Cloud Run (near-free) · is wired into DataByt at `/api/support` · answers a customer end-to-end. **You built that.**

You don't need Claude Code to follow this. Go top to bottom, pass every checkpoint, and read every error out loud. That's the whole skill. Build it.